

Figure 2: The illustration of the decoding probability $p(y_t|\cdot)$ as a 4-class classifier.

Generate-Mode: The same scoring function as in the generic RNN encoder-decoder (Bahdanau et al., 2014) is used, i.e.

$$\psi_g(y_t = v_i) = \mathbf{v}_i^\top \mathbf{W}_o \mathbf{s}_t, \quad v_i \in \mathcal{V} \cup \text{UNK} \quad (7)$$

where $\mathbf{W}_o \in \mathbb{R}^{(N+1) \times d_s}$ and $\mathbf{v}_i$ is the one-hot indicator vector for v_i .

Copy-Mode: The score for “copying” the word x_j is calculated as

$$\psi_c(y_t = x_j) = \sigma \left(\mathbf{h}_j^\top \mathbf{W}_c \right) \mathbf{s}_t, \quad x_j \in \mathcal{X} \quad (8)$$

where $\mathbf{W}_c \in \mathbb{R}^{d_h \times d_s}$, and σ is a non-linear activation function, considering that the non-linear transformation in Eq.(8) can help project $\mathbf{s}_t$ and $\mathbf{h}_j$ in the same semantic space. Empirically, we also found that using the tanh non-linearity worked better than linear transformation, and we used that for the following experiments. When calculating the copy-mode score, we use the hidden states $\{\mathbf{h}_1, \dots, \mathbf{h}_{T_S}\}$ to “represent” each of the word in the source sequence $\{x_1, \dots, x_{T_S}\}$ since the bi-directional RNN encodes not only the content, but also the location information into the hidden states in $\mathbf{M}$. The location information is important for copying (see Section 3.4 for related discussion). Note that we sum the probabilities of all x_j equal to y_t in Eq. (6) considering that there may be multiple source symbols for decoding y_t . Naturally we let $p(y_t, c|\cdot) = 0$ if y_t does not appear in the source sequence, and set $p(y_t, g|\cdot) = 0$ when y_t only appears in the source.

3.3 State Update

COPYNET updates each decoding state $\mathbf{s}_t$ with the previous state $\mathbf{s}_{t-1}$, the previous symbol y_{t-1} and the context vector $\mathbf{c}_t$ following Eq. (2) for the generic attention-based Seq2Seq model. However, there is some minor changes in the $y_{t-1} \rightarrow \mathbf{s}_t$ path for the copying mechanism. More specifically, y_{t-1} will be represented as $[\mathbf{e}(y_{t-1}); \zeta(y_{t-1})]^\top$,

where $\mathbf{e}(y_{t-1})$ is the word embedding associated with y_{t-1} , while $\zeta(y_{t-1})$ is the weighted sum of hidden states in $\mathbf{M}$ corresponding to y_t

$$\begin{aligned} \zeta(y_{t-1}) &= \sum_{\tau=1}^{T_S} \rho_{t\tau} \mathbf{h}_\tau \\ \rho_{t\tau} &= \begin{cases} \frac{1}{K} p(x_\tau, \mathbf{c}|\mathbf{s}_{t-1}, \mathbf{M}), & x_\tau = y_{t-1} \\ 0 & \text{otherwise} \end{cases} \end{aligned} \quad (9)$$

where K is the normalization term which equals $\sum_{\tau': x_{\tau'}=y_{t-1}} p(x_{\tau'}, \mathbf{c}|\mathbf{s}_{t-1}, \mathbf{M})$, considering there may exist multiple positions with y_{t-1} in the source sequence. In practice, $\rho_{t\tau}$ is often concentrated on one location among multiple appearances, indicating the prediction is closely bounded to the location of words.

In a sense $\zeta(y_{t-1})$ performs a type of read to $\mathbf{M}$ similar to the attentive read (resulting $\mathbf{c}_t$) with however higher precision. In the remainder of this paper, $\zeta(y_{t-1})$ will be referred to as *selective read*. $\zeta(y_{t-1})$ is specifically designed for the copy mode: with its pinpointing precision to the corresponding y_{t-1} , it naturally bears the location of y_{t-1} in the source sequence encoded in the hidden state. As will be discussed more in Section 3.4, this particular design potentially helps copy-mode in covering a consecutive sub-sequence of words. If y_{t-1} is not in the source, we let $\zeta(y_{t-1}) = \mathbf{0}$.

3.4 Hybrid Addressing of M

We hypothesize that COPYNET uses a hybrid strategy for fetching the content in $\mathbf{M}$, which combines both content-based and location-based addressing. Both addressing strategies are coordinated by the decoder RNN in managing the attentive read and selective read, as well as determining when to enter/quit the copy-mode.

Both the semantics of a word and its location in X will be encoded into the hidden states in $\mathbf{M}$ by a properly trained encoder RNN. Judging from our experiments, the attentive read of COPYNET is driven more by the semantics and language model, therefore capable of traveling more freely on $\mathbf{M}$, even across a long distance. On the other hand, once COPYNET enters the copy-mode, the selective read of $\mathbf{M}$ is often guided by the location information. As the result, the selective read often takes rigid move and tends to cover consecutive words, including UNKS. Unlike the explicit design for hybrid addressing in Neural Turing Machine (Graves et al., 2014; Kurach et al., 2015), COPYNET is more subtle: it provides the archi-

ture that can facilitate some particular location-based addressing and lets the model figure out the details from the training data for specific tasks.

Location-based Addressing: With the location information in $\{\mathbf{h}_i\}$, the information flow

$$\zeta(y_{t-1}) \xrightarrow{\text{update}} \mathbf{s}_t \xrightarrow{\text{predict}} y_t \xrightarrow{\text{sel. read}} \zeta(y_t)$$

provides a simple way of “moving one step to the right” on X . More specifically, assuming the selective read $\zeta(y_{t-1})$ concentrates on the ℓ^{th} word in X , the state-update operation $\zeta(y_{t-1}) \xrightarrow{\text{update}} \mathbf{s}_t$ acts as “location $\leftarrow$ location+1”, making $\mathbf{s}_t$ favor the $(\ell+1)^{th}$ word in X in the prediction $\mathbf{s}_t \xrightarrow{\text{predict}} y_t$ in copy-mode. This again leads to the selective read $\hat{h}_t \xrightarrow{\text{sel. read}} \zeta(y_t)$ for the state update of the next round.

Handling Out-of-Vocabulary Words Although it is hard to verify the exact addressing strategy as above directly, there is strong evidence from our empirical study. Most saliently, a properly trained COPYNET can copy a fairly long segment full of OOV words, despite the lack of semantic information in its $\mathbf{M}$ representation. This provides a natural way to extend the effective vocabulary to include all the words in the source. Although this change is small, it seems quite significant empirically in alleviating the OOV problem. Indeed, for many NLP applications (e.g., text summarization or spoken dialogue system), much of the OOV words on the target side, for example the proper nouns, are essentially the replicates of those on the source side.

4 Learning

Although the copying mechanism uses the “hard” operation to copy from the source and choose to paste them or generate symbols from the vocabulary, COPYNET is fully differentiable and can be optimized in an end-to-end fashion using back-propagation. Given the batches of the source and target sequence $\{X\}_N$ and $\{Y\}_N$, the objectives are to minimize the negative log-likelihood:

$$\mathcal{L} = -\frac{1}{N} \sum_{k=1}^N \sum_{t=1}^T \log \left[p(y_t^{(k)} | y_{<t}^{(k)}, X^{(k)}) \right], \quad (10)$$

where we use superscripts to index the instances. Since the probabilistic model for observing any target word is a mixture of generate-mode and copy-mode, there is no need for any additional labels for modes. The network can learn to coordinate the two modes from data. More specifically, if one particular word $y_t^{(k)}$ can be found

in the source sequence, the copy-mode will contribute to the mixture model, and the gradient will more or less encourage the copy-mode; otherwise, the copy-mode is discouraged due to the competition from the shared normalization term Z . In practice, in most cases one mode dominates.

5 Experiments

We report our empirical study of COPYNET on the following three tasks with different characteristics

1. A synthetic dataset on with simple patterns;
2. A real-world task on text summarization;
3. A dataset for simple single-turn dialogues.

5.1 Synthetic Dataset

Dataset: We first randomly generate transformation rules with 5~20 symbols and variables $\mathbf{x}$ & $\mathbf{y}$, e.g.

$$a \ b \ \mathbf{x} \ c \ d \ \mathbf{y} \ e \ f \longrightarrow g \ h \ \mathbf{x} \ m,$$

with $\{a \ b \ c \ d \ e \ f \ g \ h \ m\}$ being regular symbols from a vocabulary of size 1,000. As shown in the table below, each rule can further produce a number of instances by replacing the variables with randomly generated subsequences (1~15 symbols) from the same vocabulary. We create five types of rules, including “ $\mathbf{x} \rightarrow \emptyset$ ”. The task is to learn to do the Seq2Seq transformation from the training instances. This dataset is designed to study the behavior of COPYNET on handling simple and rigid patterns. Since the strings to repeat are random, they can also be viewed as some extreme cases of rote memorization.

Rule-type	Examples (e.g. $\mathbf{x} = i \ h \ k, \ \mathbf{y} = j \ c$)
$\mathbf{x} \rightarrow \emptyset$	$a \ b \ c \ d \ \mathbf{x} \ e \ f \rightarrow c \ d \ g$
$\mathbf{x} \rightarrow \mathbf{x}$	$a \ b \ c \ d \ \mathbf{x} \ e \ f \rightarrow c \ d \ \mathbf{x} \ g$
$\mathbf{x} \rightarrow \mathbf{x} \mathbf{x}$	$a \ b \ c \ d \ \mathbf{x} \ e \ f \rightarrow \mathbf{x} \ d \ \mathbf{x} \ g$
$\mathbf{x} \mathbf{y} \rightarrow \mathbf{x}$	$a \ b \ \mathbf{y} \ d \ \mathbf{x} \ e \ f \rightarrow \mathbf{x} \ d \ i \ g$
$\mathbf{x} \mathbf{y} \rightarrow \mathbf{x} \mathbf{y}$	$a \ b \ \mathbf{y} \ d \ \mathbf{x} \ e \ f \rightarrow \mathbf{x} \ d \ \mathbf{y} \ g$

Experimental Setting: We select 200 artificial rules from the dataset, and for each rule 200 instances are generated, which will be split into training (50%) and testing (50%). We compare the accuracy of COPYNET and the RNN Encoder-Decoder with (i.e. RNNsearch) or without attention (denoted as Enc-Dec). For a fair comparison, we use bi-directional GRU for encoder and another GRU for decoder for all Seq2Seq models, with hidden layer size = 300 and word embedding dimension = 150. We use bin size = 10 in beam search for testing. The prediction is considered

Rule-type	$x \rightarrow \emptyset$	$x \rightarrow x$	$x \rightarrow xx$	$xy \rightarrow x$	$xy \rightarrow xy$
Enc-Dec	100	3.3	1.5	2.9	0.0
RNNSearch	99.0	69.4	22.3	40.7	2.6
COPYNET	97.3	93.7	98.3	68.2	77.5

Table 1: The test accuracy (%) on synthetic data. correct only when the generated sequence is exactly the same as the given one.

It is clear from Table 1 that COPYNET significantly outperforms the other two on all rule-types except “ $x \rightarrow \emptyset$ ”, indicating that COPYNET can effectively learn the patterns with variables and accurately replicate rather long subsequence of symbols at the proper places. This is hard to Enc-Dec due to the difficulty of representing a long sequence with very high fidelity. This difficulty can be alleviated with the attention mechanism. However attention alone seems inadequate for handling the case where strict replication is needed.

A closer look (see Figure 3 for example) reveals that the decoder is dominated by copy-mode when moving into the subsequence to replicate, and switch to generate-mode after leaving this area, showing COPYNET can achieve a rather precise coordination of the two modes.

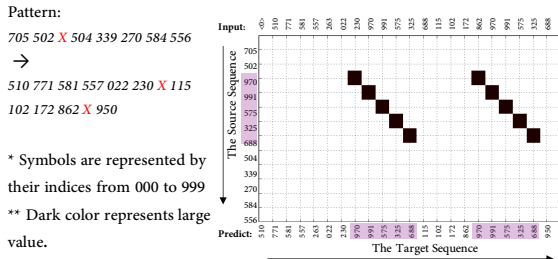


Figure 3: Example output of COPYNET on the synthetic dataset. The heatmap represents the activations of the copy-mode over the input sequence (left) during the decoding process (bottom).

5.2 Text Summarization

Automatic text summarization aims to find a condensed representation which can capture the core meaning of the original document. It has been recently formulated as a Seq2Seq learning problem in (Rush et al., 2015; Hu et al., 2015), which essentially gives *abstractive* summarization since the summary is generated based on a representation of the document. In contrast, *extractive* summarization extracts sentences or phrases from the original text to fuse them into the summaries, therefore making better use of the overall structure of the original document. In a sense, COPYNET for summarization lies somewhere between

two categories, since part of output summary is actually extracted from the document (via the copying mechanism), which are fused together possibly with the words from the generate-mode.

Dataset: We evaluate our model on the recently published LCSTS dataset (Hu et al., 2015), a large scale dataset for short text summarization. The dataset is collected from the news medias on Sina Weibo¹ including pairs of (short news, summary) in Chinese. Shown in Table 2, PART II and III are manually rated for their quality from 1 to 5. Following the setting of (Hu et al., 2015) we use Part I as the training set and the subset of Part III scored from 3 to 5 as the testing set.

Dataset	PART I	PART II	PART III
no. of pairs	2,400,591	10,666	1106
no. of score ≥ 3	-	8685	725

Table 2: Some statistics of the LCSTS dataset.

Experimental Setting: We try COPYNET that is based on character (+C) and word (+W). For the word-based variant the word-segmentation is obtained with jieba². We set the vocabulary size to 3,000 (+C) and 10,000 (+W) respectively, which are much smaller than those for models in (Hu et al., 2015). For both variants we set the embedding dimension to 350 and the size of hidden layers to 500. Following (Hu et al., 2015), we evaluate the test performance with the commonly used ROUGE-1, ROUGE-2 and ROUGE-L (Lin, 2004), and compare it against the two models in (Hu et al., 2015), which are essentially canonical Encoder-Decoder and its variant with attention.

Models		ROUGE scores on LCSTS (%)		
		R-1	R-2	R-L
RNN	+C	21.5	8.9	18.6
(Hu et al., 2015)	+W	17.7	8.5	15.8
RNN context	+C	29.9	17.4	27.2
(Hu et al., 2015)	+W	26.8	16.1	24.1
COPYNET	+C	34.4	21.6	31.3
	+W	35.0	22.3	32.0

Table 3: Testing performance of LCSTS, where “RNN” is canonical Enc-Dec, and “RNN context” its attentive variant.

It is clear from Table 3 that COPYNET beats the competitor models with big margin. Hu et al. (2015) reports that the performance of a word-based model is inferior to a character-based

¹www.sina.com

²https://pypi.python.org/pypi/jieba